File - /Users/mazenfraihat/IdeaProjects/HW10/src/test/java/part2/UserTest.java

```java
package part2;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class UserTest {

    ChatServer server = new ChatServer();
    User mazen = new User("Mazen", server);
    User ahmed = new User("Ahmed", server);

    @BeforeEach
    void setUp() {
        server.registerUser(mazen);
        server.registerUser(ahmed);
    }

    @Test
    void getName() {
        assertEquals("Mazen", mazen.getName());
    }

    @Test
    void getChatHistory() {
        assertNotNull(mazen.getChatHistory());
    }

    @Test
    void sendMessage() {
        mazen.sendMessage("hi", List.of(ahmed));
        assertEquals(1, mazen.getChatHistory().getMessages().size());
        assertEquals(1, ahmed.getChatHistory().getMessages().size());
    }

    @Test
    void receiveMessage() {
        Message message = new Message(mazen, List.of(ahmed), "hi");
        ahmed.receiveMessage(message);
        assertTrue(ahmed.getChatHistory().getMessages().contains(
message));
    }

    @Test
    void undoLastMessage() {
        mazen.sendMessage("hi", List.of(ahmed));
        mazen.undoLastMessage();
        assertTrue(mazen.getChatHistory().getMessages().isEmpty());
        assertTrue(ahmed.getChatHistory().getMessages().isEmpty());
    }
```

```
53
54      @Test
55      void blockUser() {
56          ahmed.blockUser(mazen);
57          mazen.sendMessage("hi", List.of(ahmed));
58          assertTrue(ahmed.getChatHistory().getMessages().isEmpty());
59      }
60
61      @Test
62      void viewChatHistory() {
63          mazen.sendMessage("hi", List.of(ahmed));
64          assertDoesNotThrow(() -> mazen.viewChatHistory());
65      }
66
67      @Test
68      void iterator() {
69          mazen.sendMessage("hi", List.of(ahmed));
70          assertEquals("hi", mazen.iterator(ahmed).next().getContent
    ());
71      }
72  }
```

File - /Users/mazenfraihat/IdeaProjects/HW10/src/test/java/part2/MessageTest.java

```java
package part2;

import org.junit.jupiter.api.Test;
import part1.ChatServer;
import part1.Message;
import part1.User;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class MessageTest {

    part1.ChatServer server = new ChatServer();
    part1.User mazen = new part1.User("Mazen", server);
    part1.User ahmed = new User("Ahmed", server);
    part1.Message message = new Message(mazen, List.of(ahmed), "hi");

    @Test
    void getSender() {
        assertEquals(mazen, message.getSender());
    }

    @Test
    void getRecipients() {
        assertTrue(message.getRecipients().contains(ahmed));
    }

    @Test
    void getTimestamp() {
        assertNotNull(message.getTimestamp());
    }

    @Test
    void getContent() {
        assertEquals("hi", message.getContent());
    }

    @Test
    void testToString() {
        String text = message.toString();
        assertTrue(text.contains("Mazen"));
        assertTrue(text.contains("Ahmed"));
        assertTrue(text.contains("hi"));
    }
}
```

File - /Users/mazenfraihat/IdeaProjects/HW10/src/test/java/part2/ChatServerTest.java

```java
package part2;

import org.junit.jupiter.api.Test;
import part1.ChatServer;
import part1.Message;
import part1.MessageMemento;
import part1.User;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class ChatServerTest {

    part1.ChatServer server = new ChatServer();
    part1.User mazen = new part1.User("Mazen", server);
    part1.User ahmed = new User("Ahmed", server);
    part1.Message message = new Message(mazen, List.of(ahmed), "hi");

    @Test
    void registerUser() {
        server.registerUser(mazen);
        server.registerUser(ahmed);
        server.sendMessage(message);
        assertEquals(1, ahmed.getChatHistory().getMessages().size());
    }

    @Test
    void unregisterUser() {
        server.registerUser(mazen);
        server.registerUser(ahmed);
        server.unregisterUser(ahmed);
        server.sendMessage(message);
        assertTrue(ahmed.getChatHistory().getMessages().isEmpty());
    }

    @Test
    void blockUser() {
        server.registerUser(mazen);
        server.registerUser(ahmed);
        server.blockUser(ahmed, mazen);
        assertTrue(server.isBlocked(ahmed, mazen));
    }

    @Test
    void isBlocked() {
        server.registerUser(mazen);
        server.registerUser(ahmed);
        assertFalse(server.isBlocked(ahmed, mazen));
    }

    @Test
    void sendMessage() {
```

```
54          server.registerUser(mazen);
55          server.registerUser(ahmed);
56          server.sendMessage(message);
57          assertTrue(ahmed.getChatHistory().getMessages().contains(
    message));
58      }
59
60      @Test
61      void sendMessageToBlockedUser() {
62          server.registerUser(mazen);
63          server.registerUser(ahmed);
64          server.blockUser(ahmed, mazen);
65          server.sendMessage(message);
66          assertTrue(ahmed.getChatHistory().getMessages().isEmpty());
67      }
68
69      @Test
70      void undoMessage() {
71          server.registerUser(mazen);
72          server.registerUser(ahmed);
73          server.sendMessage(message);
74          server.undoMessage(mazen, new MessageMemento("hi", message.
    getTimestamp()));
75          assertTrue(ahmed.getChatHistory().getMessages().isEmpty());
76      }
77  }
```

File - /Users/mazenfraihat/IdeaProjects/HW10/src/test/java/part2/ChatHistoryTest.java

```java
package part2;

import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class ChatHistoryTest {

    ChatServer server = new ChatServer();
    User mazen = new User("Mazen", server);
    User ahmed = new User("Ahmed", server);
    Message message = new Message(mazen, List.of(ahmed), "hi");
    MessageMemento memento = new MessageMemento("hi", message.
getTimestamp());
    ChatHistory history = new ChatHistory();

    @Test
    void addMessage() {
        history.addMessage(message);
        assertEquals(1, history.getMessages().size());
    }

    @Test
    void saveMemento() {
        history.saveMemento(memento);
        assertEquals(memento, history.getLastMessageSent());
    }

    @Test
    void getLastMessageSent() {
        assertNull(history.getLastMessageSent());
    }

    @Test
    void removeMessage() {
        history.addMessage(message);
        history.removeMessage(memento);
        assertTrue(history.getMessages().isEmpty());
    }

    @Test
    void getMessages() {
        history.addMessage(message);
        assertTrue(history.getMessages().contains(message));
    }

    @Test
    void iterator() {
        history.addMessage(message);
        assertTrue(history.iterator(ahmed).hasNext());
    }
```

```
53 }
```

File - /Users/mazenfraihat/IdeaProjects/HW10/src/test/java/part2/MessageMementoTest.java

```java
package part2;

import org.junit.jupiter.api.Test;
import part1.MessageMemento;

import java.time.LocalDateTime;

import static org.junit.jupiter.api.Assertions.assertEquals;

class MessageMementoTest {

    LocalDateTime time = LocalDateTime.now();
    part1.MessageMemento memento = new MessageMemento("hi", time);

    @Test
    void getContent() {
        assertEquals("hi", memento.getContent());
    }

    @Test
    void getTimestamp() {
        assertEquals(time, memento.getTimestamp());
    }
}
```

```java
package part2;

import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class searchMessagesByUserTest {

    ChatServer server = new ChatServer();
    User mazen = new User("Mazen", server);
    User ahmed = new User("Ahmed", server);
    User sara = new User("Sara", server);
    Message hi = new Message(mazen, List.of(ahmed), "hi");
    Message yo = new Message(sara, List.of(mazen), "yo");
    List<Message> messages = new ArrayList<>(List.of(hi, yo));

    @Test
    void hasNext() {
        searchMessagesByUser it = new searchMessagesByUser(messages, ahmed);
        assertTrue(it.hasNext());
        it.next();
        assertFalse(it.hasNext());
    }

    @Test
    void next() {
        searchMessagesByUser it = new searchMessagesByUser(messages, ahmed);
        it.hasNext();
        assertEquals(hi, it.next());
    }

    @Test
    void noMatch() {
        User omar = new User("Omar", server);
        searchMessagesByUser it = new searchMessagesByUser(messages, omar);
        assertFalse(it.hasNext());
    }
}
```